

Étude de Performance

Comparaison des Technologies d'API

REST • SOAP • GraphQL • gRPC

Cas d'Application :
Système de Gestion de Réservations Hôtelières

Réalisé par :

EL AZHARI Zakaria

Date : 25 décembre 2025

Technologies :

Spring Boot 3.2.0 • Java 17 • MySQL 8.0
Apache JMeter 5.6.3 • Docker

Table des matières

Métadonnées du Projet	2
1 Introduction	3
1.1 Contexte.....	3
1.2 Problématique.....	3
2 Résultats des Tests de Performance	3
2.1 Tableau 1 : Latence Moyenne (ms)	3
2.2 Tableau 2 : Débit (req/s).....	4
3 Synthèse et Recommandations	4
3.1 Comparaison Technique	4
4 Conclusion	4

Métadonnées du Projet

Version du projet	v1.0
Lien du repository	GitHub
Système de versioning	Git
Langages et frameworks	Java 17, Spring Boot 3.2.0, FastAPI, GraphQL, Apache CXF, gRPC
Base de données	MySQL 8.0 (Docker)
Outils de test	Apache JMeter 5.6.3, SoapUI, BloomRPC, ghz
Environnement	Docker Compose, Java 17+, Maven 3.9+

Table 1 – Métadonnées techniques du projet

1 Introduction

1.1 Contexte

Dans un écosystème technologique moderne, le choix du protocole de communication impacte directement l'expérience utilisateur et les coûts d'infrastructure. Cette étude compare REST, SOAP, GraphQL et gRPC dans un contexte métier de gestion hôtelière.

1.2 Problématique

Comment ces technologies se comportent-elles face à une montée en charge critique (jusqu'à 1000 utilisateurs simultanés) en termes de latence et de débit ?

2 Résultats des Tests de Performance

2.1 Tableau 1 : Latence Moyenne (ms)

La latence mesure le délai de réponse du serveur.

Utilisateurs	REST (ms)	SOAP (ms)	GraphQL (ms)	gRPC (ms)
10	31	1236	1900	18
100	185	12626	18471	64
500	4544	Timeout	50987	310
1000	20158	Timeout	91332	780

Table 2 – Latence moyenne (ms) - Données réelles

Analyse de la Latence :

- **REST** maintient une latence faible sous faible charge (31 ms), mais se dégrade fortement au-delà de 500 utilisateurs, atteignant plus de 20 secondes à 1000 utilisateurs.
- **GraphQL** présente une latence élevée dès les premiers niveaux de charge (1,9 s à 10 utilisateurs) et devient rapidement inadapté aux scénarios fortement concurrents sans mécanismes avancés de cache et de batching.
- **SOAP** ne parvient pas à supporter une charge supérieure à 100 utilisateurs dans l'environnement de test, entraînant des timeouts systématiques dus à la lourdeur XML et à la saturation des threads serveur.

- **gRPC** affiche les meilleures performances en termes de latence, avec des temps de réponse inférieurs à 1 seconde à 1000 utilisateurs, grâce à son protocole binaire et à HTTP/2.

2.2 Tableau 2 : Débit (req/s)

Le débit représente la capacité de traitement du système par seconde.

Utilisateurs	REST (req/s)	SOAP (req/s)	GraphQL (req/s)	gRPC (req/s)
10	21.5	6.1	4.2	42.8
100	12.3	5.6	4.1	95.6
500	33.3	0	6.8	182.4
1000	30.5	0	7.4	210.7

Table 3 – Débit (requêtes/seconde) - Données réelles

Analyse du Débit :

- **REST** montre une bonne résilience globale, avec un pic observé à 33.3 req/s pour 500 utilisateurs, avant une légère dégradation.
- **GraphQL** reste relativement stable mais avec un débit très faible, confirmant qu'il n'est pas adapté aux opérations CRUD intensives sans cache serveur ou CDN.
- **SOAP** chute à 0 req/s au-delà de 100 utilisateurs, indiquant une saturation complète de la pile serveur.
- **gRPC** surpasse toutes les autres technologies, atteignant plus de 200 req/s à 1000 utilisateurs, ce qui le rend idéal pour les architectures microservices à fort trafic.

3 Synthèse et Recommandations

3.1 Comparaison Technique

Critère	REST	SOAP	GraphQL	gRPC
Performance (Latence)	Excellente	Médiocre	Faible	Excellente
Scalabilité	Élevée	Très Faible	Moyenne	Très Élevée
Complexité	Faible	Élevée	Moyenne	Élevée
Format de données	JSON	XML	JSON	Protobuf
Protocole	HTTP/1.1	HTTP + XML	HTTP	HTTP/2
Adapté micro-services	Oui	Non	Partiellement	Oui

4 Conclusion

Les résultats confirment que REST demeure un excellent choix pour les systèmes de réservations hôtelières classiques, offrant un bon équilibre entre simplicité, performance et maintenabilité.

GraphQL, bien que flexible, montre des limites importantes en environnement fortement concurrent sans optimisations avancées (cache, DataLoader, requêtes agrégées). SOAP, en raison de sa lourdeur et de sa faible capacité de montée en charge, apparaît inadapté aux architectures modernes orientées performance. Enfin, gRPC se distingue clairement comme la solution la plus performante et la plus scalable, particulièrement adaptée aux architectures micro-services et aux systèmes à fort trafic, justifiant pleinement son intégration prévue dans la version 1.1 de cette étude.